Callback Hell, Promises, and Async/Await in JavaScript

Callback Hell

Callback Hell (also known as Pyramid of Doom) happens when multiple asynchronous operations are nested inside each other using callbacks, making code unreadable and hard to maintain.

Example of Callback Hell:

```
getUser(userId, function(user) {
  getOrders(user, function(orders) {
    getOrderDetails(orders[0], function(details) {
      processPayment(details, function(payment) {
        console.log("Payment successful!");
      });
    });
  });
});
```

Promises: Solving Callback Hell

Promises provide a cleaner way to handle asynchronous operations by allowing chaining with .then() and centralized error handling with .catch().

Promise-based Example:

```
getUser(userId)
  .then(user => getOrders(user))
  .then(orders => getOrderDetails(orders[0]))
  .then(details => processPayment(details))
  .then(() => console.log("Payment successful!"))
  .catch(err => console.error("Error:", err));
```

Async/Await: Built on Promises

Async/Await provides a synchronous-like syntax for writing asynchronous code, making it easier to read and maintain. It is built on top of Promises.

Async/Await Example:

```
async function completeOrder(userId) {
  try {
    const user = await getUser(userId);
    const orders = await getOrders(user);
    const details = await getOrderDetails(orders[0]);
    await processPayment(details);
    console.log("Payment successful!");
  } catch (err) {
    console.error("Error:", err);
  }
}
```

Differences between Promise Handling and Async/Await

Feature	Promise Handling	Async/Await

Readability	Chained with .then(), can get long	Looks like synchronous code, more readable
Error Handling	Handled with .catch()	Handled with try...catch
Syntax	Functional chaining	Imperative, sequential
Debugging	More stack traces due to chaining	Easier debugging as it looks synchronous
Parallel Execution	Use Promise.all()	Use await Promise.all()

Common Interview Questions

Q: What is Callback Hell?
A: Callback Hell is deeply nested asynchronous code using callbacks, leading to poor readability and maintainability.

Q: How do Promises solve Callback Hell?
A: Promises flatten the code using .then() chains and allow centralized error handling with .catch().

Q: What is Async/Await?
A: Async/Await is syntactic sugar over Promises that makes asynchronous code look synchronous.

Q: How is error handling different between Promises and Async/Await?
A: Promises use .catch() for errors, while Async/Await uses try...catch.

Q: Which is better: Promises or Async/Await?
A: Async/Await is generally more readable, but Promises are still useful for parallel execution with Promise.all or race conditions.